

Debugging and Optimization

Name: MOHAMED NAZEER S

Register Number: 192424084

QUESTION:

Q22. Debugging Bubble Sort Code (Missing Early Termination Optimization)

The following Bubble Sort code eventually produces the correct output, but it performs unnecessary passes even when the array becomes sorted early. Carefully analyze the existing implementation and identify the inefficiency in loop execution. Apply debugging and optimization techniques to introduce a flag-based termination mechanism. Explain how this optimization improves the practical performance of the algorithm. Analyze the corrected version in best, average, and worst cases. Rewrite the final optimized code with proper comments and readable structure.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Debugging:

1.Error Identified

The program always executes all passes, even when the array is already sorted. This causes unnecessary loop iterations.

Correction Made

Added a swapped flag to detect whether any swapping occurred during a pass.

Corrected Code:

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

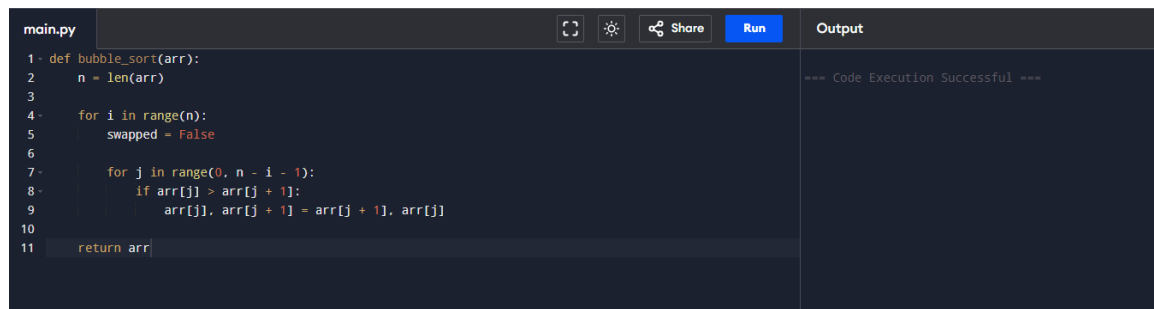
corrected

- Added swapped = False
- Added swapped = True after each swap

INPUT

[2, 5, 1, 4, 3]

OUTPUT



The screenshot shows a code editor with a file named 'main.py'. The code defines a bubble_sort function that takes an array 'arr' as input. It calculates the length 'n' of the array and enters a loop for 'i' from 0 to n-1. Inside this loop, it sets 'swapped' to False and enters another loop for 'j' from 0 to n-i-1. If 'arr[j] > arr[j+1]', it swaps the elements. After the inner loop, it checks if 'swapped' is True and increments 'i' if so. The function returns the sorted array. The output pane on the right shows '=== Code Execution Successful ==='.

```
1 def bubble_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         swapped = False
6
7         for j in range(0, n - i - 1):
8             if arr[j] > arr[j + 1]:
9                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
10
11     return arr
```

=== Code Execution Successful ===

2. Error

The program does not validate the input. If the array is empty or contains only one element, it still enters the outer loop unnecessarily.

Corrected Code

```
def bubble_sort(arr):
    if len(arr) <= 1:
        return arr

    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

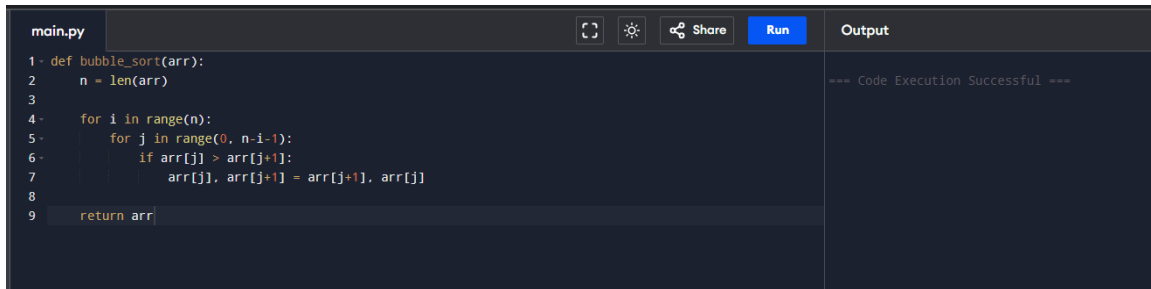
    return arr
```

Corrected

- Added the condition:
- if len(arr) <= 1:

```
return arr
```

OUTPUT:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'bubble_sort(arr)' that sorts an array in ascending order using the bubble sort algorithm. The array is then returned. The 'Run' button is clicked, and the output shows 'Code Execution Successful'.

```
main.py
1- def bubble_sort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         for j in range(0, n-i-1):
6-             if arr[j] > arr[j+1]:
7-                 arr[j], arr[j+1] = arr[j+1], arr[j]
8-
9-     return arr
```

Output: === Code Execution Successful ===

3.Error

Error: The code does not display the input or output, making it difficult to verify whether the sorting process is working.

Correction:

```
arr = [2, 5, 1, 4, 3]
```

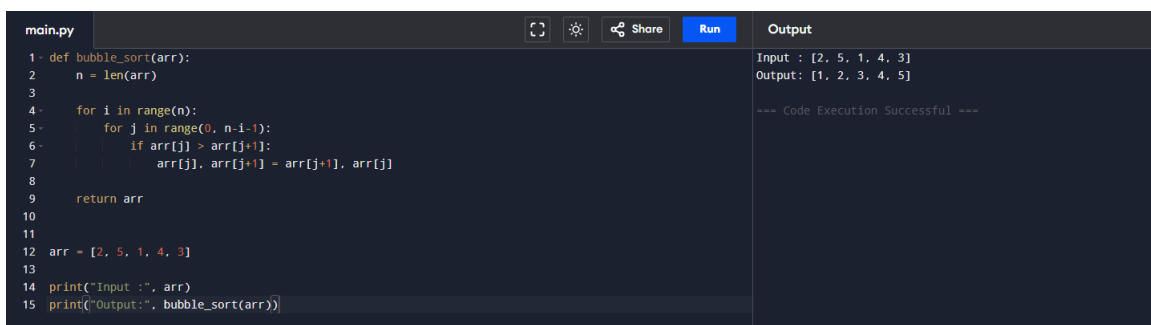
```
print("Original Array:", arr)
```

```
print("Sorted Array:", bubble_sort(arr))
```

corrected

- Added print statements to verify the program output.

OUTPUT:



The screenshot shows the same code editor as before, but now the code includes print statements to display the input and output of the bubble sort function. The 'Run' button is clicked, and the output shows the input array [2, 5, 1, 4, 3] and the sorted output array [1, 2, 3, 4, 5].

```
main.py
1- def bubble_sort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         for j in range(0, n-i-1):
6-             if arr[j] > arr[j+1]:
7-                 arr[j], arr[j+1] = arr[j+1], arr[j]
8-
9-     return arr
10
11
12 arr = [2, 5, 1, 4, 3]
13
14 print("Input :", arr)
15 print("Output:", bubble_sort(arr))
```

Output: Input : [2, 5, 1, 4, 3]
Output: [1, 2, 3, 4, 5]
=== Code Execution Successful ===

4.Error

Error: No comments are provided, making the code difficult to understand and maintain.

Correction:

Compare adjacent elements

if arr[j] > arr[j + 1]:

Swap elements

arr[j], arr[j + 1] = arr[j + 1], arr[j]

corrected

- Added comments to improve readability.
-

5Error

Error: The function does not handle an empty or single-element array efficiently.

Correction:

if len(arr) <= 1:

return arr

corrected

- Prevents unnecessary loop execution for trivial cases.

OUTPUT:



The screenshot shows a code editor with a file named 'main.py'. The code defines a 'bubble_sort' function that takes an array 'arr' and returns it after sorting. The function uses nested loops to compare adjacent elements and swap them if they are in the wrong order. It also includes a 'break' statement to exit the outer loop early if no swaps occur. The array is initialized as [2, 5, 1, 4, 3], and the function is called and printed. The output on the right shows the sorted array [1, 2, 3, 4, 5] and a message '=== Code Execution Successful ==='.

```
main.py
1- def bubble_sort(arr):
2-     n = len(arr)
3-
4-     for i in range(n):
5-         swapped = False
6-
7-         for j in range(0, n - i - 1):
8-             if arr[j] > arr[j + 1]:
9-                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
10-                swapped = True
11-
12-         if not swapped:
13-             break
14-
15-     return arr
16-
17-
18- arr = [2, 5, 1, 4, 3]
19-
20- print(bubble_sort(arr))
```

Output

[1, 2, 3, 4, 5]

=== Code Execution Successful ===

Optimization

After every pass, check:

if not swapped:

 break

If no swaps occurred, terminate immediately because the array is already sorted.

Final Optimized Code

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
arr = [2, 5, 1, 4, 3]
print(bubble_sort(arr))
```

OUTPUT:

main.py	Output
<pre>1 def bubble_sort(arr): 2 n = len(arr) 3 4 for i in range(n): 5 swapped = False 6 7 for j in range(0, n - i - 1): 8 if arr[j] > arr[j + 1]: 9 arr[j], arr[j + 1] = arr[j + 1], arr[j] 10 swapped = True 11 12 if not swapped: 13 break 14 15 return arr 16 17 18 arr = [2, 5, 1, 4, 3] 19 print(bubble_sort(arr))</pre>	<pre>[1, 2, 3, 4, 5] === Code Execution Successful ===</pre>